

Skill Session 7

Task 1

Process Synchronization and Child Process Monitoring Using waitpid()

```
j_rakshitha@RAKSHITHA: ~/o X + v
j_rakshitha@RAKSHITHA:~$ cd ~/ossp_project
nano src/waitpid_monitor.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/waitpid_monitor.c -o build/w
aitpid_monitor
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/waitpid_monitor
Process Synchronization using waitpid()
Parent PID: 1057

Parent created two children:
Child 1 started
Child 1 PID: 1058
Child 2 PID: 1059

Parent monitoring Child 1 using waitpid()...
Child 1 PID : 1058
Child 2 started
Child 1 PPID: 1057
Child 2 PID : 1059
Child 2 PPID: 1057
Child 1 completed.

Child 1 terminated with exit status: 10

Parent monitoring Child 2 using waitpid()...
Child 2 completed.

Child 2 terminated with exit status: 20

All child processes have been synchronized.
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2;
    int status;

    printf("Process Synchronization using waitpid()\n");
    printf("Parent PID: %d\n\n", getpid());

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0)
    {
        printf("Child 1 started\n");
        printf("Child 1 PID : %d\n", getpid());
        printf("Child 1 PPID: %d\n", getppid());

        sleep(2);

        printf("Child 1 completed.\n\n");
        exit(10);
    }

    child2 = fork();

    if (child2 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child2 == 0)
    {
        printf("Child 2 started\n");
        printf("Child 2 PID : %d\n", getpid());
```

```
if (child2 == 0)
{
    printf("Child 2 started\n");
    printf("Child 2 PID : %d\n", getpid());
    printf("Child 2 PPID: %d\n", getppid());

    sleep(4);

    printf("Child 2 completed.\n\n");
    exit(20);
}

printf("Parent created two children:\n");
printf("Child 1 PID: %d\n", child1);
printf("Child 2 PID: %d\n\n", child2);

printf("Parent monitoring Child 1 using waitpid()...\n");
if (waitpid(child1, &status, 0) == child1)
{
    if (WIFEXITED(status))
    {
        printf("Child 1 terminated with exit status: %d\n",
            WEXITSTATUS(status));
    }
}

printf("\nParent monitoring Child 2 using waitpid()...\n");
if (waitpid(child2, &status, 0) == child2)
{
    if (WIFEXITED(status))
    {
        printf("Child 2 terminated with exit status: %d\n",
            WEXITSTATUS(status));
    }
}

printf("\nAll child processes have been synchronized.\n");
printf("Parent process exiting.\n");

return 0;
}
```

Task 2

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/path_resolver.c

j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/path_resolver.c -o build/path_resolver
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/path_resolver
PATH Variable and Command Resolution Demo

PATH Variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0/;/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvDLISR:/mnt/c/Program Files/nod
ejs:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/;/mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/Scripts/;/mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/;/mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Launcher/;/mnt/c/Users/J Rakshitha/.local/bin:/mnt/c/Users/J Rakshitha/AppData/Local/Microso
ft/WindowsApps:/mnt/d/Coding/gcc/bin:/mnt/c/Users/J Rakshitha/AppData/Roaming/npm:/mnt/d/Coding/vs code/bin:/snap/bin

Search Directories:
- /usr/local/sbin
- /usr/local/bin
- /usr/sbin
- /usr/bin
- /sbin
- /bin
- /usr/games
- /usr/local/games
- /usr/lib/wsl/lib
- /mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe
- /mnt/c/Program Files/Common Files/Oracle/Java/javapath
- /mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath
- /mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath
- /mnt/c/Windows/system32
- /mnt/c/Windows
- /mnt/c/Windows/System32/Wbem
- /mnt/c/Windows/System32/WindowsPowerShell/v1.0/
- /mnt/c/Windows/System32/OpenSSH/
- /mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common
- /mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvDLISR
- /mnt/c/Program Files/nodejs/

- /mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/Scripts/
- /mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/
- /mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Launcher/
- /mnt/c/Users/J Rakshitha/.local/bin
- /mnt/c/Users/J Rakshitha/AppData/Local/Microsoft/WindowsApps
- /mnt/d/Coding/gcc/bin
- /mnt/c/Users/J Rakshitha/AppData/Roaming/npm
- /mnt/d/Coding/vs code/bin
- /snap/bin

Enter command to locate: ls

Command found!
Command : ls
Executable : /usr/bin/ls
Executable permission: Available
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/path_resolver
PATH Variable and Command Resolution Demo

PATH Variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0/;/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvDLISR:/mnt/c/Program Files/nod
ejs:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/;/mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/Scripts/;/mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/;/mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Launcher/;/mnt/c/Users/J Rakshitha/.local/bin:/mnt/c/Users/J Rakshitha/AppData/Local/Microso
ft/WindowsApps:/mnt/d/Coding/gcc/bin:/mnt/c/Users/J Rakshitha/AppData/Roaming/npm:/mnt/d/Coding/vs code/bin:/snap/bin

Search Directories:
- /usr/local/sbin
- /usr/local/bin
- /usr/sbin
- /usr/bin
- /sbin
- /bin
- /usr/games
- /usr/local/games
- /usr/lib/wsl/lib
- /mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe
- /mnt/c/Program Files/Common Files/Oracle/Java/javapath
- /mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath
- /mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath
- /mnt/c/Windows/system32
- /mnt/c/Windows
- /mnt/c/Windows/System32/Wbem
```

```

- /mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path
- /mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath
- /mnt/c/Windows/system32
- /mnt/c/Windows
- /mnt/c/Windows/System32/Wbem
- /mnt/c/Windows/System32/WindowsPowerShell/v1.0/
- /mnt/c/Windows/System32/OpenSSH/
- /mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common
- /mnt/c/Program Files/NVIDIA Corporation/NVIDIA NvDLISR
- /mnt/c/Program Files/nodejs/
- /mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/
- /mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/Scripts/
- /mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Python314/
- /mnt/c/Users/J Rakshitha/AppData/Local/Programs/Python/Launcher/
- /mnt/c/Users/J Rakshitha/.local/bin
- /mnt/c/Users/J Rakshitha/AppData/Local/Microsoft/WindowsApps
- /mnt/d/Coding/gcc/bin
- /mnt/c/Users/J Rakshitha/AppData/Roaming/npm
- /mnt/d/Coding/vs code/bin
- /snap/bin

```

Enter command to locate: not_a_real_command

Command not found in PATH: not_a_real_command
 Executable resolution failed.

j_rakshitha@RAKSHITHA:~/ossp_project\$ |

```

GNU nano 8.7.1 src/path_resolver.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

int main()
{
    char *path;
    char *path_copy;
    char *directory;
    char full_path[512];

    printf("PATH Variable and Command Resolution Demo\n\n");

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable is not set.\n");
        return 1;
    }

    printf("PATH Variable:\n%s\n\n", path);

    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        perror("strdup failed");
        return 1;
    }

    printf("Search Directories:\n");

    directory = strtok(path_copy, ":");

    while (directory != NULL)
    {
        printf("- %s\n", directory);
        directory = strtok(NULL, ":");
    }

    free(path_copy);
}

```

```
    return 1;
}

path_copy = strdup(path);

if (path_copy == NULL)
{
    perror("strdup failed");
    return 1;
}

directory = strtok(path_copy, ":");

while (directory != NULL)
{
    snprintf(full_path, sizeof(full_path),
             "%s/%s", directory, command);

    if (access(full_path, X_OK) == 0)
    {
        struct stat file_info;

        if (stat(full_path, &file_info) == 0)
        {
            printf("\nCommand found!\n");
            printf("Command      : %s\n", command);
            printf("Executable   : %s\n", full_path);
            printf("Executable permission: Available\n");
        }

        free(path_copy);
        return 0;
    }

    directory = strtok(NULL, ":");
}

printf("\nCommand not found in PATH: %s\n", command);
printf("Executable resolution failed.\n");

free(path_copy);

return 1;
}
```

Skill Session 8

Task 1

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/variable_expand.c
j_rakshitha@RAKSHITHA:~/ossp_project$ ls -l src/variable_expand.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2218 Aug 24 04:27 src/variable_expand.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/variable_expand.c -o
  build/variable_expand
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/variable_expand
Variable Expansion Demo
Enter text containing variables: Hello $USER

Original Input : Hello $USER
Expanded Output: Hello j_rakshitha

Variable expansion completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/variable_expand
Variable Expansion Demo
Enter text containing variables: Home directory: ${HOME}

Original Input : Home directory: ${HOME}
Expanded Output: Home directory: /home/j_rakshitha

Variable expansion completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/variable_expand
Variable Expansion Demo
Enter text containing variables: Value: $DOES_NOT_EXIST

Original Input : Value: $DOES_NOT_EXIST
Undefined variable: DOES_NOT_EXIST
Expanded Output: Value:

Variable expansion completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/variable_expand.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/variable_expand.c -o
  build/variable_expand
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/variable_expand
Variable Expansion Demo
Enter text containing variables: $MESSAGE

Original Input : $MESSAGE
Expanded Output: Hello World

Variable expansion completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

void expand_once(const char *input, char *output, int size)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < size - 1)
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;
            const char *value;

            i++;

            if (input[i] == '{')
            {
                i++;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 99)
                {
                    variable[k++] = input[i++];
                }

                if (input[i] == '}')
                    i++;
            }
            else
            {
                while (input[i] != '\0' &&
                    (isalnum((unsigned char)input[i]) ||
                     input[i] == '_') &&
                    k < 99)
                {

```

```

        {
            variable[k++] = input[i++];
        }
    }

    variable[k] = '\0';

    if (k > 0)
    {
        value = getenv(variable);

        if (value != NULL)
        {
            while (*value != '\0' && j < size - 1)
            {
                output[j++] = *value++;
            }
        }
        else
        {
            printf("Undefined variable: %s\n", variable);
        }
    }
    else
    {
        output[j++] = '$';
    }
}
else
{
    output[j++] = input[i++];
}
}

output[j] = '\0';
}

void expand_variables(const char *input, char *output, int size)
{
    char current[1000];

```

```
char current[1000];
char next[1000];

strncpy(current, input, sizeof(current) - 1);
current[sizeof(current) - 1] = '\0';

for (int i = 0; i < 5; i++)
{
    expand_once(current, next, sizeof(next));

    if (strcmp(current, next) == 0)
        break;

    strncpy(current, next, sizeof(current) - 1);
    current[sizeof(current) - 1] = '\0';
}

strncpy(output, current, size - 1);
output[size - 1] = '\0';
}

int main()
{
    char input[300];
    char output[1000];

    printf("Variable Expansion Demo\n");
    printf("Enter text containing variables: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    printf("\nOriginal Input : %s\n", input);

    expand_variables(input, output, sizeof(output));

    printf("Expanded Output: %s\n", output);

    printf("\nVariable expansion completed successfully.\n");

    return 0;
}
```

Task 2

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/builtin_shell.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/builtin_shell.c -o build/builtin_shell
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/builtin_shell
Built-in Command Shell
Available built-ins: pwd, cd, exit
Type an invalid command to test error handling.

builtin> pwd
/home/j_rakshitha/ossp_project
builtin> cd build
builtin> pwd
/home/j_rakshitha/ossp_project/build
builtin> cd ..
builtin> pwd
/home/j_rakshitha/ossp_project
builtin> hello
Unknown command: hello
Available built-ins: pwd, cd, exit
builtin> exit
Exiting shell...
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef void (*builtin_func)(char *);

void builtin_pwd(char *args)
{
    char cwd[1024];

    (void)args;

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void builtin_cd(char *args)
{
    if (args == NULL || strlen(args) == 0)
    {
        printf("cd: directory required\n");
        return;
    }

    if (chdir(args) != 0)
        perror("cd");
}

void builtin_exit(char *args)
{
    (void)args;

    printf("Exiting shell...\n");
    exit(0);
}

struct builtin
```

```
struct builtin builtins[] =
{
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"exit", builtin_exit}
};

int main()
{
    char input[1024];
    char command[100];
    char args[900];

    int builtin_count = sizeof(builtins) / sizeof(builtins[0]);

    printf("Built-in Command Shell\n");
    printf("Available built-ins: pwd, cd, exit\n");
    printf("Type an invalid command to test error handling.\n\n");

    while (1)
    {
        printf("builtin> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        command[0] = '\0';
        args[0] = '\0';

        sscanf(input, "%99s %899[^\n]", command, args);

        int found = 0;

        for (int i = 0; i < builtin_count; i++)
        {
```

```
GNU nano 8.7.1 src/builtin_shell.c *
while (1)
{
    printf("builtin> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        continue;

    command[0] = '\0';
    args[0] = '\0';

    sscanf(input, "%99s %899[^\n]", command, args);

    int found = 0;

    for (int i = 0; i < builtin_count; i++)
    {
        if (strcmp(command, builtins[i].name) == 0)
        {
            found = 1;

            builtins[i].function(args);

            break;
        }
    }

    if (!found)
    {
        printf("Unknown command: %s\n", command);
        printf("Available built-ins: pwd, cd, exit\n");
    }
}

return 0;
}
```

Practice Tasks

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser

Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
j_rakshitha@RAKSHITHA:~$ cd ~/ossp_project
nano src/syntax_parser.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/syntax_parser.c -o b
uild/syntax_parser
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/syntax_parser
Syntax Parser Demo
Enter a command: echo hello

Quote syntax: Valid

Tokens received: 2
Token 1: [echo]
Token 2: [hello]

Validating syntax...
Syntax is valid.

Parse Tree:
Command: echo
Arguments:
└─ hello

Parser completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/syntax_parser
Syntax Parser Demo
Enter a command:

Error: Empty command.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/syntax_parser
Syntax Parser Demo
Enter a command: echo "hello

Syntax Error: Unmatched quote detected.
Parsing failed.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/syntax_parser
Syntax Parser Demo
Enter a command: ls -l /home

Quote syntax: Valid

Tokens received: 3
Token 1: [ls]
Token 2: [-l]
Token 3: [/home]
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/syntax_parser
Syntax Parser Demo
Enter a command: ls -l /home

Quote syntax: Valid

Tokens received: 3
Token 1: [ls]
Token 2: [-l]
Token 3: [/home]

Validating syntax...
Syntax is valid.

Parse Tree:
Command: ls
Arguments:
├── -l
└── /home

Parser completed successfully.
```

```
GNU nano 8.7.1 src/syntax_parser.c *
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 20
#define MAX_LENGTH 100

void check_quotes(const char *input)
{
    int single_quote = 0;
    int double_quote = 0;

    for (int i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\'' && !double_quote)
            single_quote = !single_quote;

        else if (input[i] == '"' && !single_quote)
            double_quote = !double_quote;
    }

    if (single_quote || double_quote)
    {
        printf("\nSyntax Error: Unmatched quote detected.\n");
    }
    else
    {
        printf("\nQuote syntax: Valid\n");
    }
}

int main()
{
    char input[500];
    char tokens[MAX_TOKENS][MAX_LENGTH];
    int count = 0;

    printf("Syntax Parser Demo\n");
    printf("Enter a command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {

```

```
printf("Syntax Parser Demo\n");
printf("Enter a command: ");

if (fgets(input, sizeof(input), stdin) == NULL)
{
    printf("Input error.\n");
    return 1;
}

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
{
    printf("\nError: Empty command.\n");
    return 1;
}

/*
 * Check syntax BEFORE modifying the input.
 */
check_quotes(input);

/*
 * Stop parsing if quotes are unmatched.
 */
int single_quote = 0;
int double_quote = 0;

for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '\'' && !double_quote)
        single_quote = !single_quote;

    else if (input[i] == '"' && !single_quote)
        double_quote = !double_quote;
}

if (single_quote || double_quote)
{
    printf("Parsing failed.\n");
```

```
int single_quote = 0;
int double_quote = 0;

for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '\'' && !double_quote)
        single_quote = !single_quote;

    else if (input[i] == '"' && !single_quote)
        double_quote = !double_quote;
}

if (single_quote || double_quote)
{
    printf("Parsing failed.\n");
    return 1;
}

/*
 * Tokenization
 */
char *token = strtok(input, " \t");

while (token != NULL && count < MAX_TOKENS)
{
    strncpy(tokens[count], token, MAX_LENGTH - 1);
    tokens[count][MAX_LENGTH - 1] = '\0';

    count++;
    token = strtok(NULL, " \t");
}

printf("\nTokens received: %d\n", count);

for (int i = 0; i < count; i++)
{
    printf("Token %d: [%s]\n", i + 1, tokens[i]);
}

/*
 * Parse tree
```

```

/*
 * Parse tree
 */
printf("\nValidating syntax...\n");
printf("Syntax is valid.\n");

printf("\nParse Tree:\n");
printf("Command: %s\n", tokens[0]);

if (count > 1)
{
    printf("Arguments:\n");

    for (int i = 1; i < count; i++)
    {
        printf("    └─ %s\n", tokens[i]);
    }
}
else
{
    printf("Arguments: None\n");
}

printf("\nParser completed successfully.\n");

return 0;
}

```

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/single_quote_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/single_quote_demo.c
-o build/single_quote_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/single_quote_demo
Single Quote Demonstration
Single quotes preserve content literally.
Enter a single-quoted string: 'hello world'

Single quotes detected.
Quoted content: hello world
Spaces are preserved.
Variable expansion is disabled.

Single quote parsing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/single_quote_demo
Single Quote Demonstration
Single quotes preserve content literally.
Enter a single-quoted string: '$HOME'

Single quotes detected.
Quoted content: $HOME
Spaces are preserved.
Variable expansion is disabled.

Single quote parsing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
GNU nano 8.7.1 src/single_quote_demo.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[300];
    char content[300];

    printf("Single Quote Demonstration\n");
    printf("Single quotes preserve content literally.\n");
    printf("Enter a single-quoted string: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    int len = strlen(input);

    if (len >= 2 && input[0] == '\'' && input[len - 1] == '\'')
    {
        strncpy(content, input + 1, len - 2);
        content[len - 2] = '\0';

        printf("\nSingle quotes detected.\n");
        printf("Quoted content: %s\n", content);
        printf("Spaces are preserved.\n");
        printf("Variable expansion is disabled.\n");
    }
    else
    {
        printf("\nNo valid single-quoted string detected.\n");
    }

    printf("\nSingle quote parsing completed successfully.\n");

    return 0;
}
```

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/double_quote_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/double_quote_demo.c
-o build/double_quote_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/double_quote_demo
Double Quote Demonstration
Double quotes preserve spaces and allow variable expansion.
Enter a double-quoted string: "hello world"

Double quotes detected.
Spaces are preserved.
Quoted content: hello world
No variable expansion required.

Double quote parsing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/double_quote_demo
Double Quote Demonstration
Double quotes preserve spaces and allow variable expansion.
Enter a double-quoted string: "$HOME"

Double quotes detected.
Spaces are preserved.
Quoted content: $HOME
Variable $HOME detected.
Expanded value: /home/j_rakshitha

Double quote parsing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char input[300];
    char content[300];

    printf("Double Quote Demonstration\n");
    printf("Double quotes preserve spaces and allow variable expansion.\n");
    printf("Enter a double-quoted string: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    int len = strlen(input);

    if (len >= 2 && input[0] == '"' && input[len - 1] == '"')
    {
        strncpy(content, input + 1, len - 2);
        content[len - 2] = '\0';

        printf("\nDouble quotes detected.\n");
        printf("Spaces are preserved.\n");
        printf("Quoted content: %s\n", content);

        char *home = strstr(content, "$HOME");

        if (home != NULL)
        {
            char expanded[300];

            const char *value = getenv("HOME");

            if (value != NULL)
            {
                int prefix_length = home - content;
```

```

GNU nano 8.7.1 src/double_quote_demo.c *
printf("Spaces are preserved.\n");
printf("Quoted content: %s\n", content);

char *home = strstr(content, "$HOME");

if (home != NULL)
{
    char expanded[300];

    const char *value = getenv("HOME");

    if (value != NULL)
    {
        int prefix_length = home - content;

        snprintf(expanded, sizeof(expanded),
                 "%.*s%s%s",
                 prefix_length,
                 content,
                 value,
                 home + 5);

        printf("Variable $HOME detected.\n");
        printf("Expanded value: %s\n", expanded);
    }
}
else
{
    printf("No variable expansion required.\n");
}
}
else
{
    printf("\nNo valid double-quoted string detected.\n");
}

printf("\nDouble quote parsing completed successfully.\n");

return 0;
}

```

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```

j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/escape_sequence_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/escape_sequence_demo
.c -o build/escape_sequence_demo
src/escape_sequence_demo.c: In function 'main':
src/escape_sequence_demo.c:20:34: warning: comparison of integer expressions of dif
ferent signedness: 'int' and 'long unsigned int' [-Wsign-compare]
   20 |         while (input[i] != '\0' && j < sizeof(output) - 1)
      |                                   ^
src/escape_sequence_demo.c:51:27: warning: comparison of integer expressions of dif
ferent signedness: 'int' and 'long unsigned int' [-Wsign-compare]
   51 |             if (j < sizeof(output) - 1)
      |                   ^
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/escape_sequence_demo
Escape Sequence Demonstration
Supported escapes: \\, \', \", \n, \t
Enter text with escape sequences: hello\nworld

Original Input:
hello\nworld

Processed Output:
hello
world

Escape sequence processing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/escape_sequence_demo
Escape Sequence Demonstration
Supported escapes: \\, \', \", \n, \t
Enter text with escape sequences: He said "hello"

Original Input:
He said "hello"

Processed Output:
He said "hello"

Escape sequence processing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ |

```

```
#include <stdio.h>
#include <string.h>

int main()
{
    char input[300];
    char output[600];
    int i = 0;
    int j = 0;

    printf("Escape Sequence Demonstration\n");
    printf("Supported escapes: \\\", \\', \\\", \\n, \\t\n");
    printf("Enter text with escape sequences: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0' && j < sizeof(output) - 1)
    {
        if (input[i] == '\\' && input[i + 1] != '\0')
        {
            i++;

            switch (input[i])
            {
                case 'n':
                    output[j++] = '\n';
                    break;

                case 't':
                    output[j++] = '\t';
                    break;

                case '\\':
                    output[j++] = '\\';
                    break;

                case '\':
                    output[j++] = '\'';
                    break;
            }
        }
        else
        {
            output[j++] = input[i];
        }
        i++;
    }
    output[j] = '\0';

    printf("Output: %s\n", output);
}
```

```
        case '\\':
            output[j++] = '\\';
            break;

        case '\':
            output[j++] = '\';
            break;

        case '"':
            output[j++] = '"';
            break;

        default:
            output[j++] = '\\';

            if (j < sizeof(output) - 1)
                output[j++] = input[i];

            break;
    }
}
else
{
    output[j++] = input[i];
}

i++;
}

output[j] = '\\0';

printf("\nOriginal Input:\n%s\n", input);

printf("\nProcessed Output:\n%s\n", output);

printf("\nEscape sequence processing completed successfully.\n");

return 0;
}
```

Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/child_process_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/child_process_demo.c -o build/child_process_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/child_process_demo ls
Child Process Execution Demo
Parent PID: 2241
Command: ls

Parent waiting for child...
Child Process Created
Child PID: 2242
Child executing command using execvp()...

Makefile build copied.txt include sample.txt src

Child process completed.
Child exit status: 0
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/child_process_demo pwd
Child Process Execution Demo
Parent PID: 2252
Command: pwd

Parent waiting for child...
Child Process Created
Child PID: 2253
Child executing command using execvp()...

/home/j_rakshitha/ossp_project

Child process completed.
Child exit status: 0
Parent process exiting.
```

```
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/child_process_demo ls -l
Child Process Execution Demo
Parent PID: 2267
Command: ls

Parent waiting for child...
Child Process Created
Child PID: 2268
Child executing command using execvp()...

total 24
-rw-r--r-- 1 j_rakshitha j_rakshitha 164 Aug 18 04:37 Makefile
drwxr-xr-x 2 j_rakshitha j_rakshitha 4096 Aug 24 05:39 build
-rw-r--r-- 1 j_rakshitha j_rakshitha 51 Aug 18 05:21 copied.txt
drwxr-xr-x 2 j_rakshitha j_rakshitha 4096 Aug 17 05:28 include
-rw-r--r-- 1 j_rakshitha j_rakshitha 51 Aug 18 05:20 sample.txt
drwxr-xr-x 2 j_rakshitha j_rakshitha 4096 Aug 24 05:38 src

Child process completed.
Child exit status: 0
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/child_process_demo definitely_not_a_command
Child Process Execution Demo
Parent PID: 2278
Command: definitely_not_a_command

Parent waiting for child...
Child Process Created
Child PID: 2279
Child executing command using execvp()...

execvp failed: No such file or directory

Child process completed.
Child exit status: 127
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
GNU nano 8.7.1 src/child_process_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    pid_t pid;
    int status;

    if (argc < 2)
    {
        printf("Usage: %s <command> [arguments...]\n", argv[0]);
        return 1;
    }

    printf("Child Process Execution Demo\n");
    printf("Parent PID: %d\n", getpid());
    printf("Command: %s\n\n", argv[1]);

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process Created\n");
        printf("Child PID: %d\n", getpid());
        printf("Child executing command using execvp()...\n\n");

        execvp(argv[1], &argv[1]);

        perror("execvp failed");
        exit(127);
    }

    printf("Parent waiting for child...\n");
}
```

```
printf("Parent waiting for child...\n");

if (waitpid(pid, &status, 0) == -1)
{
    perror("waitpid failed");
    return 1;
}

printf("\nChild process completed.\n");

if (WIFEXITED(status))
{
    printf("Child exit status: %d\n", WEXITSTATUS(status));
}
else
{
    printf("Child did not exit normally.\n");
}

printf("Parent process exiting.\n");

return 0;
}
```